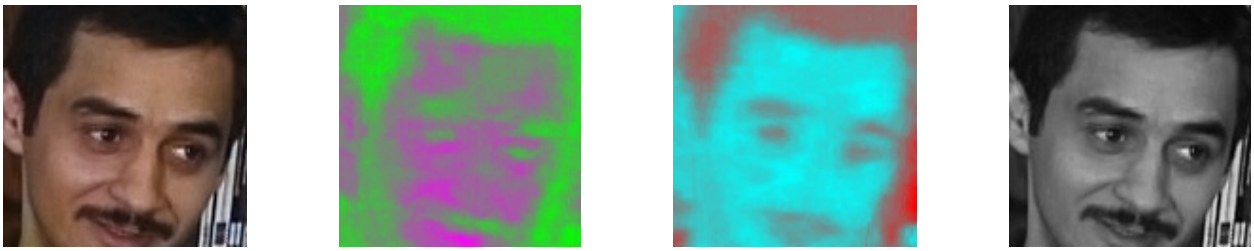
Convolutional Neural Networks For Computer Graphics Image Colorization

Hannah Qi (hq2221), Rickey Chiu (rc3868), Naythan Chan (nkc2128)



Introduction

This project implements automatic image colorization using CNNs in PyTorch. The main goal was to predict color information from grayscale images by working in the LAB color space, where the L channel represents lightness and the a and b channels represent color information.

This project was divided into four main parts: data preprocessing, a regressor, an encoder-decoder colorization network, and a transfer learning application. The models were trained and evaluated on a face image dataset, and performance was measured using MSE.

The transfer learning application was performed on a second dataset of vegetables to test whether a model pre-trained on face images could adapt to a new domain of images.

Implementation

1. Data Processing

In `step1_data.py`, we first loaded the face image dataset from `face_images/*.jpg`. The images were resized to 128 x 128 and then split into training and testing sets. 90% of the images for training and 10% for testing, so with 750 images total this gave 675 training images and 75 test images.

Then, we augmented the training images (saved in `augmented/`. For augmentation, we used random horizontal flipping, random cropping followed by resizing back to 128 x 128, and random brightness scaling. Each training image was augmented 10 times, so the final augmented training set had 6750 images. The test set was not augmented.

After that, the images were converted from RGB into LAB color space using OpenCV (saved in `lab_images`). In this color space, the L channel represents lightness and the a and b channels represent color information. The RGB tensors and the LAB tensors are saved as `.pt` files to be reused in later steps without repeating preprocessing.

2. Regressor

In `step2_regressor.py`, we implemented the simple regression model from the assignment idea. The input to the network is only the L channel, and the target is the mean value of the a and b channels across the whole image. This means the network predicts one average color for the entire image instead of predicting a full color image.

The regressor has 7 convolution layers with stride 2, which gradually reduce the spatial size from 128 x 128 down to 1 x 1. After flattening, a final linear layer outputs 2 numbers, which are the predicted mean a and mean b values. we trained this model with Adam optimizer and mean squared error loss for 20 epochs.

This model is a useful baseline, but it is limited because it cannot color different parts of the image differently. It only predicts overall average chroma values.

`step2_regressor_gpu.py` performs the same functionality as `step2_regressor.py`, but it is accelerated with GPU through the use of additional CUDA and cuNN commands after enabling CUDA use if the device is available.

3. Colorization

In `step3_colorizer.py`, we implemented a CNN colorizer that predicts the full a and b channel images from the grayscale L channel. The input is a tensor of shape $1 \times 128 \times 128$ and the output is $2 \times 128 \times 128$.

We found that using $N = 4$ instead of $N = 5$ produced better results for our colorizer.

Our final model is an encoder-decoder network. The encoder has 4 downsampling blocks, and each block uses stride 2, batch normalization, and ReLU activation. This slowly reduces the spatial resolution while increasing the number of feature channels. At the bottleneck, we apply another convolution block before decoding.

The decoder has 4 upsampling blocks. Each block upsamples the feature map by a factor of 2, then applies convolution, batch normalization, and ReLU. We also use skip connections between the encoder and decoder by concatenating intermediate encoder feature maps with decoder feature maps at matching resolutions. This helps preserve local image structure and reduces fixed-position artifacts in the output. A final 1×1 convolution maps the features to the 2 output color channels.

We trained this model using MSE loss and the Adam optimizer for 30 epochs. After training, we used the network to colorize all test images and saved the outputs of the GPU accelerated colorizer in the `GPU_colorization_results/` folder.

The `step3_colorizer_gpu.py` file performs the same functionality as `step3_colorizer.py`, but similarly to `step2_regressor_gpu.py` it is accelerated with GPU.

4. Transfer Learning

In `step5_transferlearning.py`, we reused the colorizer architecture for a transfer learning experiment on the NCD dataset. We first tuned the number of feature channels on the face dataset by testing 3 versions of the model: `n_features = 8`, `16`, and `32`. The best one was `16`, so we saved those weights as `best_face_colorizer.pt`.

Then we loaded the colored NCD dataset from `ncd_images_colored/*/*.jpg`, resized the images to 128×128, converted them to LAB color channels, and split them into train and test sets (while still using 90% training and 10% testing for the split). For this

transfer experiment, we centered the a and b channels to the range $[-1, 1]$ and used a $\tanh$ output in the network. We then loaded the pre-trained face weights and fine-tuned the full network on the colored NCD training set for 15 epochs.

Finally, we evaluated the transfer-learned model on the NCD test set and saved the predicted colorizations to `ncd_results/colorized/`. We also saved side-by-side comparison images in `ncd_results/comparisons/`. Our transfer learning step is also GPU accelerated.

Colorization Results & Analysis

Results from Colorizer for Faces Dataset

For the face dataset, the results were reasonable. The model learned skin-tone-like colors and produced face images that looked natural enough for a simple CNN.

Figure 1. recolored face

Figure 2. recolored face

Figure 3. recolored face

From looking at the saved results, the model usually produced conservative colors. A lot of the face outputs have brown or beige tones, which makes sense because many training images are faces and skin tones that are common. The results are perfect (for example there are sometimes splotches of brighter colors, such as orange, on the skin as seen most clearly in Figure 1), but they are believable.

Transfer Learning For Fruits and Vegetables Dataset

For the transfer learning experiment on the NCD dataset, the outputs were also close to the targets. Some example comparison files are:

Figure 4. grayscale, model prediction, ground truth

Figure 6. grayscale, model prediction, ground truth

Figure 7. grayscale, model prediction, ground truth

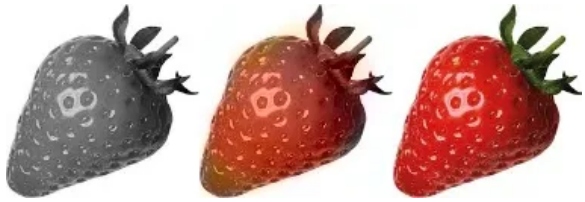


Figure 8. grayscale, model prediction, ground truth

In these comparison images, the predicted image is relatively close to the original image. There are some inconsistencies in the coloration (for example, green appearing on part of the tomato in Figure 7), but for the most part it is representative of the main colors of the ground truth image. One reason the transfer-learning MSE became extremely small may be that many NCD examples are objects with low color variation or close-to-neutral colors, so the prediction task may be easier.

Numerical Results

The main **mean squared error** (MSE) results from our experiments are:

- **Step 2 regressor baseline:** final test MSE was approximately 0.0002.
- **Step 3 colorizer (local/CPU run):** final test MSE was 0.000405.
- **Step 3 colorizer (GPU run):** test MSE improved to about 0.0003 by the end of training.

For the transfer learning and hyperparameter tuning experiment in Step 5, we tested three versions of the face colorizer:

- `n_features = 8`: face test MSE = 0.001881

- `n_features = 16`: face test MSE = 0.001924
- `n_features = 32`: face test MSE = 0.001616

The best architecture for transfer learning was therefore the model with `n_features = 32`, with best face test MSE of 0.001616.

After loading those pretrained face weights and fine-tuning on the colored NCD dataset, the final NCD test MSE was:

- **Transfer learning on NCD:** final test MSE = 0.011840

We also recorded summary statistics for the predicted and target chroma values on the NCD test set:

- **Predicted centered a/b values:**
  mean = 0.0895, standard deviation = 0.1595, min 0.3022, max = 0.6269
- **Target centered a/b values:**
  mean = 0.0901, standard deviation = 0.1791, min = 0.4353 max = 0.7020

These statistics show that the predicted chroma distribution was fairly close to the target distribution, although the prediction range was somewhat narrower than the true range.

Speedup Gained with GPU Implementation (EC)

After implementing the GPU versions of both the regressor and the colorizer, we compared the runtime against the CPU versions. The GPU versions ended up being much faster.

For the regressor:

- CPU runtime: $1\,\text{min}\ \ 1.577s$
- GPU runtime: $0\,\text{min}\ \ 14.854s$

This gives a speedup of about $4.1\times$.

For the colorizer:

- CPU runtime: $15\,\text{min}\ \ 46.448s$
- GPU runtime: $2\,\text{min}\ \ 2.395s$

This gives a speedup of about $7.7\times$.

These results show that moving the CNN training to the GPU gave a clear performance improvement, especially for the larger colorizer model. This is expected because CNNs involve many parallel tensor operations, which GPUs handle much more efficiently than CPUs.

Tanh Output Experiment (EC)

For the extra credit experiment, we created a separate GPU implementation called `step3_colorizer_gpu_extracredit.py` that uses a $\tanh$ activation at the output of the network. To make this work correctly, we first rescaled the target a and b channels from the range $[0, 1]$ to $[-1, 1]$. Then, after the model predicted the output, we converted the predicted values back to $[0, 1]$ before reconstructing and saving the LAB image.

The pros of using $\tanh$ is that the a and b chroma channels is bounded and can take both positive and negative deviations around a neutral center. This is a more natural match for color channels than leaving the output unconstrained.

Overall, the outputted colored image was similarly acceptable, differing with a notable brighting of skin tone for the $\tanh$ implemented version. Therefore, the change was reasonable for the design but did not dramatically improve performance for this dataset.

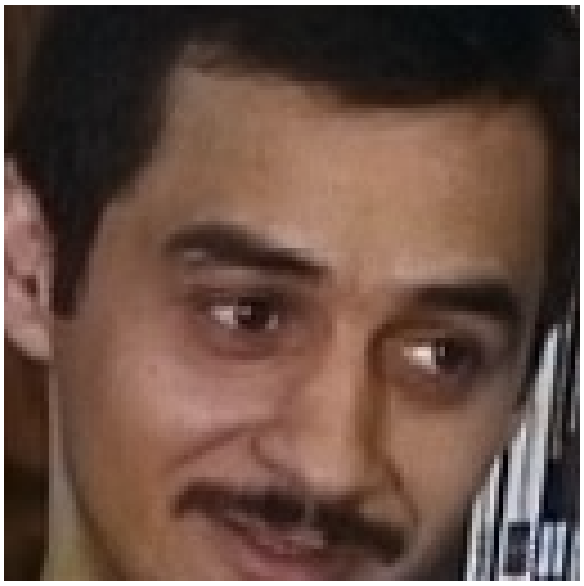


Figure 9. output with original activation function

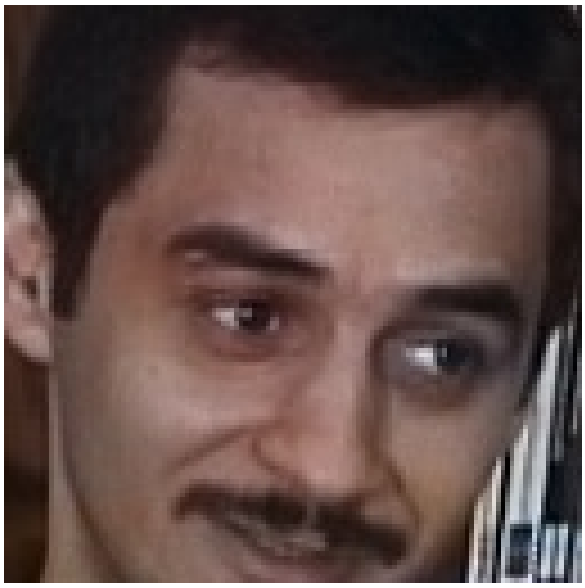


Figure 10. output with $\tanh$ activation function

Challenges and Difficulties

One difficulty in this project was understanding how OpenCV stores LAB values, because OpenCV uses an encoded version of LAB instead of the exact textbook ranges. Because of that, we normalized the stored LAB channels consistently and used the same representation during training and reconstruction.

Another challenge was that low MSE does not always mean the output looks visually perfect. Some outputs still looked a little dull even when the numerical error was small. We think this is because MSE tends to encourage average-looking colors instead of stronger or more saturated predictions.

We also ran into an architectural issue in Step 3. In the earlier version of our colorizer, the image was compressed too aggressively through repeated downsampling before being reconstructed. This created a bottleneck where too much local spatial detail was lost, and as a result the model sometimes produced regions of unsaturated color. To fix this, we changed the model to a U-Net style encoder-decoder with skip connections, so the decoder could reuse spatial information from earlier layers instead of reconstructing everything from a very small bottleneck representation.

During implementation, another important issue we resolved was that the original `ncd_images/` folder contained grayscale versions of the fruit and vegetable images. Since those images had no real color information in their a and b channels, the model would only learn to predict gray output. To fix this, we renamed that folder to `ncd_images_grayscale/` and used the original color images in `ncd_images_colored/` for transfer learning instead.

Running Our Code

1. Create and activate a conda environment
   `conda create -n nameofenv python=3.9`
   `conda activate nameofenv`
2. Install dependencies:
   `pip install -r requirements.txt`
3. Put the face dataset images into:
   `face_images/`
4. Run preprocessing and data augmentation:
   `python step1_data.py`

This creates:

- `train_tensor.pt` , `test_tensor.pt` , `train_lab.pt` , `test_lab.pt`
- augmented image files
- LAB visualization folder (`lab_images`)
  - inside the `lab_images` folder there are three separate folders: `L` , `a` , `b`

5. Train the regressor:
   `python step2_regressor.py` or `python step2_regressor_gpu.py`

   This creates: `regressor.pt`

6. Train the colorizer and generate results:
   `python step3_colorizer.py` or `python step3_colorizer_gpu.py`

   This creates: `colorizer.pt` and colorized image results in `GPU_colorization_results/`

7. For transfer learning, put the NCD dataset in:
   `ncd_images_colored` for the colored images zip file
   `ncd_images_grayscale` for the grayscale image zip file

   Then run:
   `python step5_transferlearning.py`

   This creates:

   - `best_face_colorizer.pt`
   - `transfer_colorizer_ncd.pt`
   - colorized NCD results in `ncd_results/colorized/`
   - comparison images in `ncd_results/comparisons/`